File Edit View Run Kernel Tabs Settings Help

Day_10_Task_5.ipynb | Day11_Task_2.ipynb | Day11_Task.ipynb | sample_student_performanc

Code | Python 3 (ipykernel)

[1]:
```
import pandas as pd
import numpy as np

df = pd.read_csv("ecommerce_dirty_sample.csv")
```

[2]:
```
df.columns
df[["product_name", "category", "unit_price", "total_amount"]].head()
df["unit_price"].info()
```

```
<class 'pandas.Series'>
RangeIndex: 104 entries, 0 to 103
Series name: unit_price
Non-Null Count  Dtype
--------------  -----
102 non-null    float64
dtypes: float64(1)
memory usage: 964.0 bytes
```

[3]:
```
df["unit_price"] = pd.to_numeric(df["unit_price"], errors="coerce")
```

[4]:
```
df["unit_price"].isnull().sum()
df[df["unit_price"].isnull()]
df[df["unit_price"] < 0]
```

[4]:

	order_id	order_date	customer_id	product_name	category	quantity	unit_price	total_amount	payment_method	order_status	city
64	ORD1065	2026-03-08	CUST065	Coffee Maker	Home Appliances	2	-45.0	90.0	UPI	Delivered	Pune

[5]:
```
max_price = df["unit_price"].max()
print("Highest product price:", max_price)
```

```
Highest product price: 999999.99
```

[6]:
```
highest_price_record = df[df["unit_price"] == max_price]
highest_price_record
```

[6]:

	order_id	order_date	customer_id	product_name	category	quantity	unit_price	total_amount	payment_method	order_status	city
35	ORD1036	2026-02-07	CUST036	Running Shoes	Fashion	1	999999.99	79.99	Debit Card	Shipped	Delhi

[7]:
```
highest_price_record[
    ["order_id", "order_date", "customer_id", "product_name",
     "category", "quantity", "unit_price", "total_amount",
     "payment_method", "order_status", "city"]
]
```

[7]:

	order_id	order_date	customer_id	product_name	category	quantity	unit_price	total_amount	payment_method	order_status	city
35	ORD1036	2026-02-07	CUST036	Running Shoes	Fashion	1	999999.99	79.99	Debit Card	Shipped	Delhi

[8]:
```
print("Maximum price:", df["unit_price"].max())
print("Price in selected record:", highest_price_record["unit_price"].iloc[0])
print("Product name:", highest_price_record["product_name"].iloc[0])
print("Category:", highest_price_record["category"].iloc[0])
print("Order ID:", highest_price_record["order_id"].iloc[0])

assert highest_price_record["unit_price"].max() == max_price
```

```
Maximum price: 999999.99
Price in selected record: 999999.99
Product name: Running Shoes
Category: Fashion
Order ID: ORD1036
```

[9]:
```
print("Total records:", len(df))
print("Missing prices:", df["unit_price"].isnull().sum())
print("Negative prices:", (df["unit_price"] < 0).sum())
print("Highest valid numeric price:", df["unit_price"].max())
```

```
Total records: 104
Missing prices: 2
Negative prices: 1
Highest valid numeric price: 999999.99
```

[10]:
```
df["unit_price"].dropna().nlargest(2)
```

[10]:
```
35    999999.99
0        899.99
Name: unit_price, dtype: float64
```

Would you like to get notified about official Jupyter news?
Open privacy policy Yes No

Would you like to get notified about official Jupyter news?
Open privacy policy Yes N

Would you like to get notified about official Jupyter news?
Open privacy policy Yes N